

🐍 Python Learning Notes – Part 1 (Basics)

1. Why Programming is Important?

Imagine you have a robot at home. If you don't give it **instructions**, it will just sit quietly. Programming is the way we **instruct computers** to do tasks for us.

- Without programming → Computer is just a box.
- With programming → It can calculate, play music, control rockets, run websites like Google/Instagram.

☞ Example:

You want to add two numbers manually: $10 + 20 = 30$.

But if you tell the computer:

```
print(10 + 20)
```

It instantly gives **30**. That's the power of programming – **speed, accuracy, automation**.

2. Program vs Coding

- **Coding** → Writing individual instructions (like words in a sentence).
- **Program** → A complete set of instructions to solve a real problem (like a full story).

☞ Example:

```
# Coding (single instruction)
print("Hello")
```

```
# Program (multiple steps to greet user)
name = input("Enter your name: ")
print("Hello", name)
```

Here, just writing `print("Hello")` is **coding**.

But combining **input + output + logic** becomes a **program**.

3. History of Python

- Created by **Guido van Rossum** in **1991**.
- He wanted a language that is **simple like English** but also **powerful**.
- Named after a comedy TV show “Monty Python’s Flying Circus” (not the snake 🐍).

☞ Today, Python is everywhere:

- Google, YouTube, Instagram → use Python.

- AI & Machine Learning → Python is #1 choice.
 - Data Science → Python + Pandas/Numpy.
 - Web Development → Python + Django/Flask.
-

4. Advantages of Python

Why is Python so popular compared to C, C++ or Java?

- ✓ Easy syntax (like English).
- ✓ Free & Open-source.
- ✓ Works on all OS (Windows/Linux/Mac).
- ✓ Huge community → thousands of libraries.
- ✓ Supports many styles of programming (procedural, OOP, functional).
- ✓ Used in **AI, Data Science, Web Dev, Automation, Gaming, IoT**.

🔗 Example (Python vs Java code):

```
# Python: Simple & short
print("Hello World")
```

```
# Java: Longer
class Hello {
    public static void main(String args[]) {
        System.out.println("Hello World");
    }
}
```

5. Programming Paradigms (Styles of Writing Code)

(a) Procedural Programming

- Write code step by step (like cooking recipe).
- Focus is on **functions**.

```
def square(x):
    return x * x

print(square(5))    # 25
```

(b) Object-Oriented Programming (OOP)

- Organizes code into **objects** (like real-world things).
- Example: Car has properties (color, speed) and methods (start, stop).

```
class Car:
```

```
def __init__(self, brand):
    self.brand = brand

def drive(self):
    print(self.brand, "is driving")

c = Car("Tesla")
c.drive()    # Tesla is driving
```

(c) Functional Programming

- Focuses on **functions only**.
- Avoids changing data (immutability).
- Uses **lambda, map, filter, reduce**.

```
nums = [1, 2, 3, 4]
squared = list(map(lambda x: x*x, nums))
print(squared)    # [1, 4, 9, 16]
```

6. Framework vs Product vs Language

- **Language** → The tool we use to write code (Python, Java).
- **Framework** → Pre-made helper code to make apps faster (Django, Flask).
- **Product** → Final app built using framework + language (Instagram, YouTube).

☞ Example:

- **Language:** Python
 - **Framework:** Django
 - **Product:** Instagram
-

7. Python vs Framework

- Python → A general programming language.
- Framework → A library built on top of Python for specific tasks.

☞ Example:

- **Python** alone → You can write any program.
 - **Flask (framework)** → You can quickly build a website without writing everything from scratch.
-

8. Python Architecture (How Python Runs Your Code)

When you run a `.py` file:

1. **Python Source Code** (your code)
2. Compiled into **Bytecode** (.pyc file)
3. Executed by **Python Virtual Machine (PVM)**

This makes Python **portable** → the same code runs on any computer.

🐍 Python Learning Notes – Part 2 (Core Fundamentals)

9. Variables & Data Types

A **variable** is like a **container** that stores data in memory.

📖 Example:

```
x = 10          # integer
y = 3.14        # float (decimal)
name = "Mk"     # string
is_ok = True    # boolean
```

Python automatically detects type → you don't need to declare `int` or `string` like in C/Java.

✓ **Common Data Types:**

- `int` → whole numbers (10, -5)
 - `float` → decimal numbers (3.14, -2.5)
 - `str` → text ("hello")
 - `bool` → True / False
 - `list`, `tuple`, `set`, `dict` → (we'll see later in detail).
-

10. Variable Scope

Scope = where a variable can be used.

- **Global Variable** → defined outside any function.
- **Local Variable** → defined inside a function.

☞ Example:

```
x = "global"

def test():
    x = "local"
    print("Inside function:", x)

test()
print("Outside function:", x)
```

◆ Output:

```
Inside function: local
Outside function: global
```

11. Typecasting & Type Checking

Sometimes we need to convert one type into another.

☞ Example:

```
a = "10"
b = int(a)    # convert string → int
print(b + 5)  # 15

print(type(b)) # <class 'int'>
```

✓ Useful functions:

- `int()` → to integer
 - `float()` → to float
 - `str()` → to string
-

12. Operators in Python

Operators are symbols that perform operations.

☞ Example:

```
a, b = 5, 2

# Arithmetic
print(a + b) # 7
print(a - b) # 3
print(a * b) # 10
print(a / b) # 2.5
print(a // b) # 2 (floor division)
print(a % b) # 1 (remainder)
print(a ** b) # 25 (power)

# Comparison
print(a > b) # True
print(a == b) # False
```

```
# Logical
print(a > 2 and b > 1) # True
print(a > 10 or b > 1) # True
```

13. Comments in Python

Comments = Notes in code (ignored by Python).

☞ Example:

```
# This is a single-line comment

"""
This is
a multi-line
comment
"""
```

Use comments to **explain your logic**.

14. Input Handling

We take user input with `input()`.

☞ Example:

```
name = input("Enter your name: ")
age = int(input("Enter your age: ")) # convert to int
print("Hello", name, "you are", age, "years old.")
```

✓ Always remember: `input()` returns a **string**, so typecast if needed.

15. String Handling

Strings = text inside " " or ' '.

☞ Example:

```
s = "Python"
print(len(s))      # length
print(s.upper())   # PYTHON
print(s.lower())   # python
print(s[0])        # first character 'P'
print(s[0:3])      # slicing 'Pyt'
print(s[::-1])     # reverse 'nohtyP'
```

✓ Strings are **immutable** → you can't change characters directly.

16. Conditional Statements

Used for **decision making**.

☞ Example:

```
x = 15

if x > 20:
    print("Greater than 20")
elif x == 15:
    print("Equal to 15")
else:
    print("Less than 20")
```

☞ Output: Equal to 15

17. Loops in Python

(a) For Loop

Used to repeat actions.

```
for i in range(5):    # 0 to 4
    print(i)
```

☞ Output: 0 1 2 3 4

(b) While Loop

Runs until condition is False.

```
x = 5
while x > 0:
    print(x)
    x -= 1
```

☞ Output: 5 4 3 2 1

✓ Use **break** (stop loop) and **continue** (skip one step) inside loops.

18. List, Tuple, Set, Dictionary

(a) List → ordered, changeable

```
lst = [1, 2, 3, 4]
lst.append(5)
print(lst[0])    # 1
```

(b) Tuple → ordered, unchangeable

```
tup = (1, 2, 3)
print(tup[1])    # 2
```

(c) Set → unordered, unique values

```
st = {1, 2, 2, 3}
print(st) # {1, 2, 3}
```

(d) Dictionary → key-value pairs

```
dic = {"a": 1, "b": 2}
print(dic["a"]) # 1
```

🌀 Python Learning Notes – Part 3 (Intermediate Concepts)

19. OOP Concepts (Object-Oriented Programming)

OOP = A way of organizing code like **real-world objects**.

Each object has **properties (data)** and **methods (actions)**.

☞ Core OOP Concepts in Python:

1. **Class** → Blueprint (like a design of a car).
 2. **Object** → Actual thing created from a class (like a real car).
 3. **Encapsulation** → Binding data + methods together.
 4. **Inheritance** → One class can use features of another.
 5. **Polymorphism** → Same function name but different behavior.
 6. **Abstraction** → Hide internal details, show only important things.
-

Example: OOP in Python

```
# Class
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color

    def drive(self):
        print(self.brand, "is driving")

# Object
c1 = Car("Tesla", "Red")
c2 = Car("BMW", "Black")
```



```
c1.drive()    # Tesla is driving
c2.drive()    # BMW is driving
```

Inheritance Example

```
class Animal:
    def sound(self):
        print("Some sound")

class Dog(Animal):    # Dog inherits from Animal
    def sound(self):
        print("Bark")

d = Dog()
d.sound()    # Bark
```

☞ Here, **Dog** reuses **Animal**'s blueprint but also **overrides** the sound.

20. File Handling in Python

Python allows us to **read & write files**.

Writing to a file:

```
with open("myfile.txt", "w") as f:
    f.write("Hello Python\n")
```

Reading from a file:

```
with open("myfile.txt", "r") as f:
    data = f.read()
    print(data)
```

Appending to a file:

```
with open("myfile.txt", "a") as f:
    f.write("New line added\n")
```

✓ "w" = write (overwrites), "a" = append, "r" = read.

21. Exception Handling

Errors may crash a program. We handle them using **try-except**.

☞ Example:

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Error: Cannot divide by zero")
finally:
    print("This always runs")
```

⚡ Output:

Error: Cannot divide by zero
This always runs

✓ finally block is optional → it runs no matter what.

22. Database Connection (SQLite Example)

Python has `sqlite3` module to connect to **databases**.

📄 Example:

```
import sqlite3

# Connect to DB (creates file if not exists)
conn = sqlite3.connect("test.db")

# Create cursor (to run queries)
cur = conn.cursor()

# Create table
cur.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER, name TEXT)")

# Insert data
cur.execute("INSERT INTO users (id, name) VALUES (?, ?)", (1, "Mk"))

# Commit changes
conn.commit()

# Fetch data
cur.execute("SELECT * FROM users")
print(cur.fetchall())  # [(1, 'Mk')]

# Close connection
conn.close()
```

✓ Database stores data permanently (not lost after program ends).

✓ You can use MySQL, PostgreSQL, etc. with similar logic.

🐍 Python Learning Notes – Part 4 (Functional Programming)

23. Higher-Order Functions

A **higher-order function** = a function that can:

1. Take another function as input, OR
2. Return another function.

☞ Example:

```
def greet():  
    return "Hello"  
  
def call_func(func):  
    return func()  
  
print(call_func(greet))    # Hello
```

24. Pure vs Impure Functions

- **Pure Function** → Always gives same output for same input, has no side effects.
- **Impure Function** → Depends on external state, may change data.

☞ Example:

```
# Pure function  
def pure_add(a, b):  
    return a + b  
  
# Impure function  
count = 0  
def impure_increment():  
    global count  
    count += 1  
    return count
```

✓ Pure functions are easier to test and debug.

25. Lambda Functions (Anonymous Functions)

A **lambda** is a small, one-line function without `def`.

☞ Example:

```
square = lambda x: x * x  
print(square(5))    # 25
```

✓ Useful when you need quick functions for `map`, `filter`, etc.

26. Map, Filter, Reduce

(a) map() → Apply function to every item.

```
nums = [1, 2, 3, 4]  
squares = list(map(lambda x: x*x, nums))  
print(squares)    # [1, 4, 9, 16]
```

(b) filter() → Keep only items that satisfy condition.

```
nums = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens) # [2, 4, 6]
```

(c) reduce() → Reduce list into a single value.

```
from functools import reduce
nums = [1, 2, 3, 4]
product = reduce(lambda a, b: a * b, nums)
print(product) # 24
```

27. Closure

A **closure** = a function inside another function that remembers the outer variable.

☞ Example:

```
def outer(x):
    def inner(y):
        return x + y
    return inner

add5 = outer(5) # inner function remembers x=5
print(add5(10)) # 15
```

28. Partial Functions

Use `functools.partial` to **fix some arguments** of a function.

☞ Example:

```
from functools import partial

def power(base, exp):
    return base ** exp

square = partial(power, exp=2)
print(square(5)) # 25
```

29. Function Composition

Combine two or more functions into one.

☞ Example:

```
def f(x): return x + 2
```

```
def g(x): return x * 3

def compose(f, g):
    return lambda x: f(g(x))

h = compose(f, g)
print(h(4)) # g(4)=12 → f(12)=14
```

30. Callback Functions

A function passed into another function to be executed later.

☞ Example:

```
def process(data, callback):
    print("Processing:", data)
    callback()

process("Task 1", lambda: print("Done!"))
```

31. Recursive Function

A function that calls itself.

☞ Example: Factorial

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)

print(factorial(5)) # 120
```

✓ Recursive functions must have a **base case** to avoid infinite loops.

32. Generator & Yield

- **Generator** = a function that produces values one by one using `yield`.
- Saves memory compared to lists.

☞ Example:

```
def gen_numbers():
    for i in range(3):
        yield i

for num in gen_numbers():
    print(num)
```

⚡ Output:

0

1
2

✓ Normal function → returns all values at once.

✓ Generator → produces values on demand.

🌀 Python Learning Notes – Part 5 (Advanced Topics)

33. Pytest (Testing in Python)

When programs get big, we need to **test** if functions are working correctly.

☞ **pytest** is a framework for writing and running tests.

Example:

```
# mycode.py
def add(a, b):
    return a + b

# test_mycode.py
from mycode import add

def test_add():
    assert add(2, 3) == 5
```

Run in terminal:

```
pytest test_mycode.py
```

✓ If test passes → code works.

✓ If test fails → you know where the bug is.

34. Scheduling with Python

We can **automate tasks** using the `schedule` library.

☞ Example:

```
import schedule, time

def job():
    print("Task running...")

schedule.every(2).seconds.do(job)
```

```
while True:
    schedule.run_pending()
    time.sleep(1)
```

⚡ Output:

```
Task running...
Task running...
```

(prints every 2 seconds).

35. Tkinter GUI (Graphical User Interface)

Tkinter = Python's built-in **GUI toolkit**.

📄 Example:

```
import tkinter as tk

root = tk.Tk()
root.title("My First App")

label = tk.Label(root, text="Hello Tkinter!")
label.pack()

root.mainloop()
```

✓ This opens a small window with text “Hello Tkinter!”.

36. Streamlit (Data Science Web Apps)

Streamlit = quick way to make **data apps**.

📄 Example:

```
import streamlit as st

st.title("Welcome to Streamlit")
name = st.text_input("Enter your name:")
if st.button("Say Hello"):
    st.write("Hello", name)
```

Run with:

```
streamlit run app.py
```

✓ Opens in browser → interactive app!

37. Matplotlib (Data Visualization)

Matplotlib = library for **graphs & charts**.

☞ Example:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [2, 4, 6, 8]

plt.plot(x, y, marker="o")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Simple Graph")
plt.show()
```

✓ This shows a line graph.

38. Python Logging

Instead of just using `print()`, we can log info for debugging.

☞ Example:

```
import logging

logging.basicConfig(level=logging.INFO)

logging.info("Program started")
logging.warning("This is a warning")
logging.error("This is an error")
```

✓ Useful when apps grow big → logs help track errors.

39. Pandas (Data Handling)

Pandas = powerful library for working with **tables (like Excel)**.

☞ Example:

```
import pandas as pd

data = {"Name": ["Mk", "Ram", "Sam"], "Age": [22, 25, 28]}
df = pd.DataFrame(data)

print(df)
print(df["Name"])      # select column
print(df.describe())   # quick stats
```

✓ Widely used in **Data Science & AI**.

40. Flask (Web Development)

Flask = lightweight web framework.

☞ Example:


```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello Flask!"

if __name__ == "__main__":
    app.run(debug=True)
```

✓ Run → opens a local website showing *Hello Flask!*

41. REST API (Web APIs with Flask)

APIs let apps talk to each other.

🔗 Example:

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/api/data")
def get_data():
    return jsonify({"name": "Mk", "msg": "Hello API"})

if __name__ == "__main__":
    app.run(debug=True)
```

✓ Run → visit <http://127.0.0.1:5000/api/data> → you get JSON data.

42. Functional Programming Tips

- ✓ Prefer **pure functions** (easy to test).
 - ✓ Use **map/filter/reduce** instead of loops when possible.
 - ✓ Avoid changing data → work with **copies**.
 - ✓ Use **lambda** for quick functions.
 - ✓ Combine small functions → cleaner code.
-